ENGINEERING BRIEF · AGENT C · NEW ONBOARDING

Roomio — Multi-Room & Connectors

Welcome aboard, Agent C. Your domain is "the house": expand Roomio from a single room to multiple connected rooms — kitchen, bathroom, dining, and more — joined by architecturally correct connectors (doorways, cased openings, archways, pass-throughs, hallways). Everything optional for the user to add. You coordinate with Agents A and B through the shared log.

MULTI-ROOM / FLOOR PLAN | CONNECTORS | ALL OPTIONAL

Repo: `~/Desktop/personal/roomio` · **Your folder:** `/multi-room`
Coordination: `/shared/*.json` · **Joint log:** `roomio.txt`
Date: June 29, 2026 · **Companion to:** Room Designer v2, Agent A & B briefs

Contents

1 Onboarding: Who You Are, Who A & B Are 03

2 Your Mission & the "Everything Optional" Rule 04

3 The Room-Type Taxonomy + Essentials 05

4 The Connector Taxonomy 07

5 Adjacency Rules (which connector fits where) 09

6 Data Schemas (house / room / connector) 10

7 Coordination with A & B + User Channel 11

8 Functional Requirements & Acceptance 12

9 Build Milestones 13

1. Onboarding: Who You Are, Who A & B Are

> You are **Agent C**, the third Claude instance on Roomio, joining a project already in motion. Three agents share one repo and coordinate through `roomio.txt` (the joint log) and `/shared` (the contract surface). Read `roomio.txt` end-to-end first to absorb the project's history and the other agents' current state before writing any code.

Agent	Domain	What they own
Agent A	Front-end / room interior	The room-design UI: shape wizard, furnishing, archetype library, persona presets, design-suggestion engine. "Inside one room."
Agent B	Detection + camera	Local furniture-detection pipeline (Ollama/YOLO) and the virtual-camera flythrough feature. Standalone services + scene integration.
Agent C (you)	The house / floor plan	Multiple rooms and the connectors between them. "The whole dwelling." You sit a level *above* A's single-room work.

MENTAL MODEL OF THE LAYERING

A designs *inside* a room. You arrange *rooms into a house* and connect them. B observes/films. Your floor-plan structure should be the container that A's room-interiors plug into — so design your house/room schema to wrap, not replace, A's existing single-room model.

2. Your Mission & the "Everything Optional" Rule

Today Roomio builds **one room**. Your mission: let a user assemble **multiple rooms** of different types into a connected floor plan, and join them with **architecturally appropriate connectors**. Kitchen and bathroom are explicitly in scope as room types, alongside living, dining, bedroom, and others.

THE "EVERYTHING OPTIONAL" RULE (CORE CONSTRAINT)
Nothing you add is mandatory. The user is never *forced* to add a kitchen, a bathroom, a hallway, or any connector. A single bedroom (today's Roomio) must remain a complete, valid design. Every room type and every connector is an **opt-in** the user chooses to place. Your features expand what's *possible*, never what's *required*.

Practically, that means:

- The multi-room canvas starts from (or wraps) a single room and lets the user *add* more rooms when they want.
- Connectors are placed deliberately by the user between two rooms; the app may *suggest* a sensible connector (per §5) but never auto-inserts a required one.
- Kitchen/bathroom come with helpful per-type essentials and layout guidance (§3), offered as assistance, not enforced.

RESEARCH-GROUNDED, NOT INVENTED
Like the other agents, ground your room-type essentials, connector types, and adjacency rules in documented architectural sources (floor-plan standards, NKBA-style kitchen/bath guidance). This brief seeds them; verify and extend via research before encoding. Don't invent clearances or rules from intuition — real numbers exist.

3. The Room-Type Taxonomy + Essentials

Each room type carries a default purpose, typical essentials, and layout guidance. Essentials are **offered** (and can feed A's suggestion engine), never forced. Seeded from design sources; verify/extend.

Room type	Essentials (offered)	Layout guidance (research-seeded)
Bedroom	Bed, nightstand(s), wardrobe, lamp	Bed as focal point; nightstand + bedside lighting pairing; clearance to walk both sides where possible.
Living room	Sofa, coffee table, rug, media unit, lighting	Conversation zone with seating facing a focal point; rug anchors seating; layered lighting; don't wall-hug.
Kitchen	Counters/cabinets, sink, stove/cooktop, refrigerator, optional island	Classic **work triangle** (sink–stove–fridge): each leg ~4–9 ft, total ~13–26 ft, no major traffic crossing it. Modern alt: **zone** planning (storage→prep→wash→cook, one-direction flow). Aisle clearance min ~42 in (one cook), ~48 in (two). Island only if room ≥ ~12 ft wide; keep island ≥ ~42 in from appliances. Layouts: galley, L, U, island, peninsula.
Bathroom	Toilet, sink/vanity, shower and/or bathtub, optional storage	Plumbing fixtures along wet walls; maintain code-style clearances in front of each fixture; door swing must not hit fixtures; ventilation/privacy from public rooms.
Dining room	Dining table, chairs, optional sideboard, light over table	Allow ~32 in from wall to table edge where no one passes behind diners, ~44 in if traffic passes behind; pendant centered over table.
Home office	Desk, chair, shelving, task light	Desk oriented to light/window without screen glare; task + ambient lighting; clear path to seat.
Entry / foyer	Console, coat storage, landing surface	Transitional space; framed sight line into the next room; landing spot for keys/mail.
Hallway	(circulation; minimal furniture)	Connector-as-room; keep width comfortable for passage; can host the connectors that branch to other rooms.

USE THE CORPUS; REQUEST WHAT'S MISSING

Kitchen and bathroom need fixtures Roomio likely hasn't modeled yet (counters, sink, stove, fridge, toilet, tub, shower). Use the closest archetype or placeholder box where absent, and log `REQUEST -> ASSET` entries in `roomio.txt` for the missing fixtures so they get modeled. Don't block on missing assets.

4. The Connector Taxonomy

Connectors are how two rooms join. They are real architectural elements with distinct looks and rules — this is the part to get right. Each connector occupies a span on a **shared wall** between two rooms (or opens a room to a hallway). Seeded from floor-plan and architectural sources; verify/extend.

Connector	What it is	Typical use / rules
Hinged doorway	Standard single swinging door in an opening.	Default room-to-room/private connector. Has a swing arc — the swing must stay clear of fixtures/ furniture. Direction matters.
Double door	Two hinged leaves side by side.	Wider, more formal openings (e.g. into a dining or primary suite). Two swing arcs.
Pocket / sliding door	Door that slides into/along the wall.	Space-saving where a swing won't fit; adds privacy without arc. Pocket needs wall depth to receive it.
Cased opening	A framed doorway-sized opening with *no door*.	Connects spaces while keeping definition; no privacy. Common kitchen↔dining, hall↔living.
Archway	A curved-top open passage (shown in plan as a dashed header line).	Decorative open connector; warmth/character; joins living↔dining, foyer↔living. No door.
Wide / double-wide opening	An extra-wide cased/framed opening.	Open-plan feel between major living spaces; strong sight line. Wide span on the shared wall.
Pass-through	A window-like opening in a wall above counter height.	Classic kitchen↔dining/living serving hatch; opens sight/serving line without a full doorway.
Half-wall / pony wall	A partial-height wall dividing without enclosing.	Defines zones while keeping openness; living↔dining, stair edges.
Hallway link	A circulation room that connects several rooms.	Use when many rooms branch off a common path; each room then connects to the hallway via a doorway/opening.

CONNECTOR MECHANICS TO IMPLEMENT

A connector lives on a **shared wall segment** between two rooms: it has a position along that wall, a width, a type, and (for doors) a swing direction/arc. Placing one cuts the opening in both rooms' walls so the spaces actually join. Reuse Agent A's existing door/window-on-wall logic where possible rather than reinventing wall-cutting.

5. Adjacency Rules (which connector fits where)

Not every connector suits every room pairing. These rules drive *suggestions* (never hard blocks) so the floor plan stays sensible. Seeded from architectural adjacency principles; verify/extend.

ID	Pairing / condition	Suggest / rationale
AC1	Bathroom ↔ any room	Suggest a **hinged or pocket door** (privacy). Discourage open connectors (cased opening, archway, pass-through) — bathrooms need to close.
AC2	Kitchen ↔ dining	Suggest **cased opening, wide opening, or pass-through** — open flow for serving; door optional.
AC3	Living ↔ dining	Suggest **archway, wide opening, or half-wall** — open-plan continuity with definition.
AC4	Bedroom ↔ any	Suggest a **hinged or pocket door** (privacy). Open connectors discouraged.
AC5	Foyer/entry ↔ living	Suggest **archway or cased opening** — welcoming framed sight line.
AC6	Many rooms branching off one area	Suggest inserting a **hallway** link so rooms don't all open into each other awkwardly.
AC7	Door swing overlaps fixtures/furniture/ another door	Warn and suggest flipping swing, switching to pocket/sliding, or relocating the connector.
AC8	Two rooms placed adjacent with no connector	Suggest adding a connector on the shared wall (so the rooms are actually reachable) — but the user may decline.
AC9	Connector width below a sensible passage minimum	Suggest widening for comfortable passage.

SUGGESTIONS, NOT LAWS

Mirror Agent A's suggestion-engine philosophy: these are dismissible nudges with a one-line rationale, never blocks. A user can put an archway into a bathroom if they insist — you warn, you don't forbid.

6. Data Schemas (house / room / connector)

Design the house schema to **wrap** A's existing single-room model, so a one-room design is just a house with one room and no connectors.

```
{
  "house_id": "h_001",
  "rooms": [
    {
      "room_id": "r_living",
      "type": "living",                // bedroom|living|kitchen|bathroom|dining|office|foyer|hallway
      "footprint": { "shape": "rectangular", "x": 0, "z": 0, "w": 480, "l": 400 },
      "interior": { /* A's existing single-room model: walls, materials, placed items */ }
    },
    { "room_id": "r_kitchen", "type": "kitchen", "footprint": {..}, "interior": {..} }
  ],
  "connectors": [
    {
      "connector_id": "c_1",
      "type": "cased_opening",         // hinged|double|pocket|cased_opening|archway|wide_opening|pass_through
      "between": ["r_living", "r_kitchen"],
      "shared_wall": { "room_a_wall": 2, "room_b_wall": 0 },
      "position_along_wall": 0.5,       // 0..1
      "width_cm": 120,
      "swing": null                     // {dir, hinge} for doors; null for open connectors
    }
  ]
}
```

BACKWARD COMPATIBILITY IS MANDATORY

A saved single-room design from today must still open correctly — load it as a one-room house with an empty connectors array. Coordinate the exact `interior` shape with Agent A via `/shared` so you wrap their model rather than fork it.

7. Coordination with A & B + the User Channel

Onboard, then communicate (roomio.txt)

- **First action:** read `roomio.txt` in full to learn project history and A/B status. Then post an onboarding entry introducing yourself and your domain.
- Append-only; prefix every entry `[YYYY-MM-DD HH:MM] [AGENT-C]`.
- Re-read `roomio.txt` at the start of every work cycle; treat entries from A, B, or the user as high-priority.
- Log INTENT before big changes, DECISIONS (schema, connector set), and explicit `REQUEST -> AGENT-A` / `REQUEST -> AGENT-B` / `REQUEST -> ASSET`.

Watch for direct user messages to you

> The user may address you directly in `roomio.txt` — look specifically for entries tagged to you, e.g. `[USER -> AGENT-C]` or `[USER]` lines about rooms/connectors. Treat any direct user instruction as **top priority**, above your current plan. Acknowledge it in your next entry and act on it. Scan for a user channel every cycle.

Key cross-agent dependencies

- → **Agent A:** agree the `interior` room-model shape so your house wraps it; reuse A's wall door/window-cutting logic for connectors; feed room-type essentials into A's suggestion engine. Post a `REQUEST -> AGENT-A` to align schemas.
- → **Agent B:** a multi-room house means camera paths and detection may span rooms later — publish your house schema to `/shared` so B can extend flythroughs across connectors when ready.
- **Isolation:** your code in `/multi-room`; only other writable places are `/shared` and `roomio.txt`; commit only your own paths to avoid git collisions.

8. Functional Requirements & Acceptance

ID	Requirement	Area
M-1	User can add multiple rooms to a single floor plan and position them.	Multi-room
M-2	Rooms can be typed (bedroom, living, kitchen, bathroom, dining, office, foyer, hallway).	Room types
M-3	Kitchen offers work-triangle/zone guidance + fixture essentials (offered, not forced).	Kitchen
M-4	Bathroom offers fixture set + clearance/privacy guidance (offered, not forced).	Bathroom
M-5	User can place connectors on shared walls between rooms; opening is cut in both rooms.	Connectors
M-6	All connector types supported: hinged, double, pocket/sliding, cased opening, archway, wide opening, pass-through, half-wall, hallway link.	Connectors
M-7	Doors carry a swing direction/arc; swing-overlap warns (AC7).	Connectors
M-8	Adjacency rules suggest sensible connectors per pairing; all dismissible, none blocking.	Adjacency
M-9	Everything is optional: a single-room design remains complete and valid; nothing forced.	Core rule
M-10	House schema wraps A's room model; today's single-room saves still load.	Compatibility
M-11	All code in `/multi-room`; coordinate via `/shared` + `roomio.txt`; no edits to A/B source.	Isolation
M-12	Reads `roomio.txt` each cycle incl. direct user messages; logs intent/ decisions/requests.	Comms

Acceptance test (the "done" bar)

Start from a single bedroom (today's Roomio) — it still works untouched. Add a living room beside it; place a cased opening on the shared wall and watch both rooms open to each other. Add a kitchen; it offers work-triangle guidance and fixture essentials. Add a bathroom; placing an archway into it triggers a privacy warning suggesting a door instead (dismissible). Add a dining room and connect living↔dining with an archway. A door whose swing hits a fixture warns. Save and reload the whole multi-room house; also confirm an old single-room file still opens.

9. Build Milestones

Milestone	Deliverable	Definition of done
C0	Read roomio.txt fully; post onboarding entry; scaffold `/multi-room`; align house/room/connector schema with Agent A	Onboarded; schemas in `/shared`; A pinged for interior-model shape.
C1	Multi-room canvas: add/position multiple rooms; wrap A's room model	Two+ rooms coexist; old single-room file loads.
C2	Room typing + per-type essentials (incl. kitchen/bathroom)	Rooms can be typed; essentials offered per type.
C3	Connectors on shared walls (start: hinged door + cased opening), opening cut in both rooms	Two rooms genuinely join via a placed connector.
C4	Full connector set + door swing/arc + swing-overlap warning	All connector types placeable; swing logic works.
C5	Adjacency suggestion rules (AC1–AC9), dismissible	Sensible connector suggestions per pairing; privacy warnings fire.
C6	Kitchen work-triangle/zone + bathroom clearance guidance; asset requests; polish	Kitchen/bathroom guidance live; missing fixtures requested in roomio.txt.

> **One-line summary.** Expand Roomio from one room to a connected multi-room house in `/multi-room`: typed rooms (incl. kitchen & bathroom with research-grounded essentials), a full taxonomy of architecturally correct connectors on shared walls, and dismissible adjacency suggestions — all strictly optional, wrapping Agent A's room model, and coordinated with A, B, and the user through `roomio.txt`.

End of brief · Roomio Multi-Room & Connectors (Agent C) · Verify all clearances, connector types, and adjacency rules against architectural sources before encoding.